SpringAI实战



一、环境搭建

1.1 软件环境

- Java 17
- Node.js 18+
- MySQL 8
- DashScope API KEY（或者其他）
- Redis-Stack：redis基础上拓展向量查询功能

1.2 pom依赖

- 快速搭建springboot3+springAI
- 注意：使用阿里千问时，屏蔽掉其他的ai引入

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-
4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.4.1</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>com.hxx</groupId>
    <artifactId>springAI-one</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>springAI-one</name>
    <description>springAI-one</description>
    <url/>
    <licenses>
        <license/>
    </licenses>
    <developers>
        <developer/>
    </developers>
    <scm>
        <connection/>
        <developerConnection/>
        <tag/>
        <url/>
    </scm>
    <properties>
```

```
        <java.version>17</java.version>
        <spring-ai.version>1.0.0-M2</spring-ai.version>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <!--openAI
        <dependency>
            <groupId>org.springframework.ai</groupId>
            <artifactId>spring-ai-openai-spring-boot-starter</artifactId>
        </dependency>-->

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-devtools</artifactId>
            <scope>runtime</scope>
            <optional>true</optional>
        </dependency>
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <optional>true</optional>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
        <!--通义千问-->
        <dependency>
            <groupId>com.alibaba.cloud.ai</groupId>
            <artifactId>spring-ai-alibaba-starter</artifactId>
            <version>${spring-ai.version}</version>
        </dependency>
        <!--智谱清言
        <dependency>
            <groupId>org.springframework.ai</groupId>
            <artifactId>spring-ai-zhipuai-spring-boot-starter</artifactId>
        </dependency>-->
        <!--千帆：文心一言
        <dependency>
            <groupId>org.springframework.ai</groupId>
            <artifactId>spring-ai-qianfan-spring-boot-starter</artifactId>
        </dependency>-->
        <!-- kimi接入
        <dependency>
            <groupId>org.springframework.ai</groupId>
            <artifactId>spring-ai-moonshot-spring-boot-starter</artifactId>
        </dependency>
        -->
    </dependencies>
    <dependencyManagement>
        <dependencies>
            <dependency>
                <groupId>org.springframework.ai</groupId>
                <artifactId>spring-ai-bom</artifactId>
                <version>${spring-ai.version}</version>
                <type>pom</type>
                <scope>import</scope>
            </dependency>
        </dependencies>
    </dependencyManagement>

    <build>
```

```
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <configuration>
                    <annotationProcessorPaths>
                        <path>
                            <groupId>org.projectlombok</groupId>
                            <artifactId>lombok</artifactId>
                        </path>
                    </annotationProcessorPaths>
                </configuration>
            </plugin>
            <plugin>
                <groupId>org.graalvm.buildtools</groupId>
                <artifactId>native-maven-plugin</artifactId>
            </plugin>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
                <configuration>
                    <excludes>
                        <exclude>
                            <groupId>org.projectlombok</groupId>
                            <artifactId>lombok</artifactId>
                        </exclude>
                    </excludes>
                </configuration>
            </plugin>
        </plugins>
    </build>
    <repositories>
        <repository>
            <id>spring-milestones</id>
            <name>Spring Milestones</name>
            <url>https://repo.spring.io/milestone</url>
            <snapshots>
                <enabled>false</enabled>
            </snapshots>
        </repository>
    </repositories>
</project>
```

- 使用deepseek配置：操作代码相同

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-
4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.4.1</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>com.hxx</groupId>
    <artifactId>deepseek-demo</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>deepseek-demo</name>
    <description>deepseek-demo</description>
    <url/>
    <licenses>
        <license/>
    </licenses>
    <developers>
        <developer/>
    </developers>
    <scm>
        <connection/>
```

```
        <developerConnection/>
        <tag/>
        <url/>
    </scm>
    <properties>
        <java.version>17</java.version>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-devtools</artifactId>
            <scope>runtime</scope>
            <optional>true</optional>
        </dependency>
        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <optional>true</optional>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>

        <!-- Spring AI-->
        <dependency>
            <groupId>org.springframework.ai</groupId>
            <artifactId>spring-ai-openai-spring-boot-starter</artifactId>
        </dependency>
    </dependencies>

    <dependencyManagement>
        <dependencies>
            <dependency>
                <groupId>org.springframework.ai</groupId>
                <artifactId>spring-ai-bom</artifactId>
                <version>1.0.0-M5</version>
                <type>pom</type>
                <scope>import</scope>
            </dependency>
        </dependencies>
    </dependencyManagement>

    <repositories>
        <repository>
            <id>spring-milestones</id>
            <name>Spring Milestones</name>
            <url>https://repo.spring.io/milestone</url>
            <snapshots>
                <enabled>false</enabled>
            </snapshots>
        </repository>
        <repository>
            <id>spring-snapshots</id>
            <name>Spring Snapshots</name>
            <url>https://repo.spring.io/snapshot</url>
            <releases>
                <enabled>false</enabled>
            </releases>
        </repository>
    </repositories>
    <build>
        <plugins>
```

```
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <configuration>
                    <annotationProcessorPaths>
                        <path>
                            <groupId>org.projectlombok</groupId>
                            <artifactId>lombok</artifactId>
                        </path>
                    </annotationProcessorPaths>
                </configuration>
            </plugin>
            <plugin>
                <groupId>org.graalvm.buildtools</groupId>
                <artifactId>native-maven-plugin</artifactId>
            </plugin>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
                <configuration>
                    <excludes>
                        <exclude>
                            <groupId>org.projectlombok</groupId>
                            <artifactId>lombok</artifactId>
                        </exclude>
                    </excludes>
                </configuration>
            </plugin>
        </plugins>
    </build>

</project>
```

- application.yaml

```
spring:
  ai:
    openai:
      api-key: xxxxxx
      base-url: https://api.deepseek.com
      chat:
        options:
          model: deepseek-chat
```

- 配置类

```
package com.hxx.common;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.datatype.jsr310.JavaTimeModule;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class JacksonConfig {

   @Bean
   public ObjectMapper objectMapper() {
       ObjectMapper objectMapper = new ObjectMapper();
       objectMapper.registerModule(new JavaTimeModule());
       return objectMapper;
   }
}
```

1.3 配置文件

```yaml
spring:
  ai:
    # 阿里灵积
    dash-scope:
      api-key: xxx
      chat:
        options:
          model: qwen-max
    # 智谱清言
    zhipuai:
      api-key: xxx
      chat:
        enabled: true
        options:
          model: GLM-4
    # 百度千帆
    qian-fan:
      api-key: xxx
      secret-key: xxx
    # kimi
    moonshot:
      api-key: sk-G5ybErbhcqRPmyoD83BtemeOvC6vfoYyPXSgrWAlfrVQc662
```

1.4 对应的api模型

1.4.1 阿里灵积

```
private final ChatModel chatModel;
```

1.4.2 智谱清言

```
private final ZhiPuAiChatModel zhiPuAiChatModel;
```

1.4.3 百度千帆

```
private final QianFanChatModel qianFanChatModel;
```

1.4.4 Kimi接入

```
private final MoonshotChatModel moonshotChatModel;
```

二、入门AI

- 创建一个springboot3项目
- 使用阿里的灵积模型

2.1 基础案例

2.1.1 消息发送

```
package com.hxx.controller;

import com.fasterxml.jackson.databind.ObjectMapper;
import lombok.SneakyThrows;
import org.springframework.ai.chat.client.ChatClient;
import org.springframework.ai.chat.messages.SystemMessage;
import org.springframework.ai.chat.messages.UserMessage;
import org.springframework.ai.chat.model.ChatModel;
import org.springframework.ai.chat.model.ChatResponse;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.MediaType;
import org.springframework.http.codec.ServerSentEvent;
```

```java
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import reactor.core.publisher.Flux;

@RestController
public class QinwenController {

    @Autowired
    private ChatModel chatModel;
    @Autowired
    private ObjectMapper objectMapper;

    /**
     * 处理GET请求的聊天接口
     *
     * 该方法使用预定义的聊天模型来响应用户的聊天请求它通过传入用户的消息作为提示来生成回复
     * 主要涉及与ChatClient的交互，利用其prompt方法构建聊天提示，调用call方法生成回复，并提取回复的内容
     *
     * @param prompt 用户发送的聊天消息，作为聊天模型的输入提示
     * @return 聊天模型生成的回复内容
     */
    @GetMapping("/chat")
    public String chat(String prompt){
        // 创建ChatClient实例，用于与聊天模型交互
        ChatClient chatClient = ChatClient.create(chatModel);
        // 使用ChatClient实例的prompt方法开始构建聊天提示，通过user方法添加用户消息，call方法发送提示并接收回复，最后提取回复的内容返回
        return chatClient.prompt()
                .user(prompt)
                .call()
                .content();
    }

    /**
     * 处理聊天流请求
     * 该方法使用Server-Sent Events (SSE)技术，允许服务器向客户端推送实时更新
     * 在这种情况下，它用于创建一个聊天流，其中服务器根据用户的提示实时生成消息
     *
     * @param prompt 用户的输入提示，用于生成聊天响应
     * @return 返回一个Flux对象，该对象会生成一系列ServerSentEvent，每个事件包含一个聊天响应
     */
    @GetMapping(value = "/chat/stream", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
    public Flux<ServerSentEvent<String>> chatSSE(String prompt){
        // 创建一个ChatClient实例，用于与聊天模型进行交互
        // 这里体现了函数式编程的思想，通过链式调用构建复杂的交互逻辑
        return ChatClient.create(chatModel)
                .prompt()
                .messages(new SystemMessage("你好，帮助用户解决编程问题"),new UserMessage(prompt))
                .stream()
                .chatResponse()
                // 将每个聊天响应映射为一个ServerSentEvent对象
                // 使用map函数进行流的转换，体现了函数式编程的转换操作
                .map(chatResponse -> ServerSentEvent.<String>builder(toJson(chatResponse))
                        .event("message")
                        .build());
    }

    @SneakyThrows
    public String toJson(ChatResponse chatResponse) {
        try {
            return objectMapper.writeValueAsString(chatResponse);
        }catch (Exception e){
            e.printStackTrace();
        }
        return null;
    }
}
```

2.1.2 对话内存

- MessageChatMemoryAdvisor
- MessageChatMemoryAdvisor`会读取会话id对应的消息列表，并把消息列表拼接到历史的消息中。
- 历史消息的实现过程：
  - 提供ChatMemory，通过会话id查找历史消息。下面的例子提供的是`InMemoryChatMemory`，内存存储。
  - MessageChatMemoryAdvisor`会在运行时调用`ChatMemory`，通过会话id查找历史消息。然后把消息列表拼接到历史的消息中。
  - 发送消息给大模型得到答案
- 案例

```
package com.hxx.controller;

import com.fasterxml.jackson.databind.ObjectMapper;
import lombok.SneakyThrows;
import org.springframework.ai.chat.client.ChatClient;
import org.springframework.ai.chat.client.advisor.MessageChatMemoryAdvisor;
import org.springframework.ai.chat.memory.ChatMemory;
import org.springframework.ai.chat.memory.InMemoryChatMemory;
import org.springframework.ai.chat.messages.SystemMessage;
import org.springframework.ai.chat.messages.UserMessage;
import org.springframework.ai.chat.model.ChatModel;
import org.springframework.ai.chat.model.ChatResponse;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.MediaType;
import org.springframework.http.codec.ServerSentEvent;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import reactor.core.publisher.Flux;

@RestController
public class QinwenController1 {

    @Autowired
    private ChatModel chatModel;

    private ChatMemory chatMemory = new InMemoryChatMemory();

    /**
     * 根据会话id，从数据库中查找历史消息，并将消息作为上下文回答。
     *
     * @param prompt    用户的提问
     * @param sessionId 会话id
     * @return SSE流响应
     */
    @GetMapping(value = "chat/stream/history", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
    public Flux<ServerSentEvent<String>> chatStreamWithHistory(@RequestParam String prompt,
                                                               @RequestParam String sessionId) {
        // 1. 如果需要存储会话和消息到数据库，自己可以实现ChatMemory接口，
        //    这里使用InMemoryChatMemory，内存存储。
        // 2. 传入会话id，MessageChatMemoryAdvisor会根据会话id去查找消息。
        // 3. 只需要携带最近10条消息
        var messageChatMemoryAdvisor = new MessageChatMemoryAdvisor(chatMemory, sessionId, 10);
        return ChatClient.create(chatModel).prompt()
                .user(prompt)
                // MessageChatMemoryAdvisor会在消息发送给大模型之前，从ChatMemory中获取会话的历史消息，
                // 然后一起发送给大模型。
                .advisors(messageChatMemoryAdvisor)
                .stream()
                .content()
                .map(chatResponse -> ServerSentEvent.builder(chatResponse)
                        .event("message")
                        .build());
    }
}
```

2.1.3 嵌入模型

- 如何用一段话去数据库查找数据？答案是向量求相似度。就像cos求两个向量之间的夹角，cos值越大，夹角越小，夹角越小，相似度越高。所以第一件事情需要将句子向量化，然后和向量数据库中的向量进行比较，找到最相似的向量。
- 嵌入模型就可以帮我们做到这件事情。
- 选择模型

```yaml
spring:
  ai:
    # 阿里灵积
    dash-scope:
      api-key: sk-xxxxx
      chat:
        options:
          model: qwen-max
      embedding:
        options:
          model: text-embedding-v2
```

- 案例

```java
package com.hxx.controller;

import com.alibaba.cloud.ai.dashscope.api.DashScopeApi;
import org.springframework.ai.chat.client.ChatClient;
import org.springframework.ai.chat.client.advisor.MessageChatMemoryAdvisor;
import org.springframework.ai.chat.memory.ChatMemory;
import org.springframework.ai.chat.memory.InMemoryChatMemory;
import org.springframework.ai.chat.model.ChatModel;
import org.springframework.ai.embedding.EmbeddingModel;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.MediaType;
import org.springframework.http.codec.ServerSentEvent;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import reactor.core.publisher.Flux;

import java.util.Arrays;

@RestController
public class QinwenController2 {

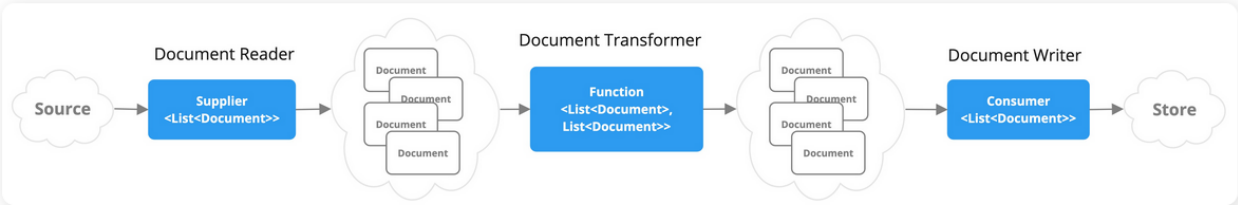
    // 阿里嵌入模型
    @Autowired
    private EmbeddingModel embeddingModel;

    @GetMapping(value = "/embedding", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
   public String getEmbedding(String text){
        // 获取嵌入向量
        float[] embed = embeddingModel.embed(text);
        return Arrays.toString(embed);
    }
}
```

2.1.4 ETL Pipeline

- ETL是提取、转换、加载的缩写，从原始的文档到数据库需要经历提取（.doc、.ppt、.xlsx等）、转换（数据结构化、清理数据、数据分块）、写入向量数据库。这个过程可以进行多种处理，确保最后的数据适合AI问答。
- SpringAI提供了ETL框架。它是搭建知识库框架的基石。
- 框架介绍：
    - DocumentReader：文档读取器，读取文档，比如PDF、Word、Excel等。如：JsonReader（读取JSON）,TextReader（读取文本）,PagePdfDocumentReader（读取PDF）,TikaDocumentReader（读取各种文件，大部分都可以支持.pdf,.xlsx,.docx,.pptx,.md,.json等）。上诉的这些reader都是DocumentReader的实现类。

- DocumentTransformer：文档转换器，处理文档。TextSplitter（文档切割成小块），ContentFormatTransformer（将文档变成键值对），SummaryMetadataEnricher（使用大模型总结文档），KeywordMetadataEnricher（使用大模型提取文档关键词）。
- DocumentWriter: 文档写入器，将文档写入向量数据库或者本地文件。VectorStore（向量数据库写入器），FileDocumentWriter（文件写入器）。



ETL 流水线结构图

- pom

```
    <dependency>
        <groupId>org.springframework.ai</groupId>
        <artifactId>spring-ai-tika-document-reader</artifactId>
    </dependency>
```

2.1.4.1 读取文件

- `TikaDocumentReader`比较全能大部分文件都可以读取，支持的文件格式可以参考[官方文档](https://tika.apache.org/2.9.0/formats.htm。如果是比较个性化文档的场景，最好自己实现一个`Reader`，比如我们公司的有些excel里面存了各种的表格数据这种只能自己去读取处理。使用读取器读出来的数据乱乱的。
  - DocumentReader会从Resource中读取文件。
  - Resource是SpringIO中的一个接口，它表示一个资源，可以是文件、网络资源、内存资源等。
- 案例

```
package com.hxx.controller;

import lombok.SneakyThrows;
import org.springframework.ai.document.Document;
import org.springframework.ai.reader.tika.TikaDocumentReader;
import org.springframework.core.io.FileSystemResource;
import org.springframework.core.io.InputStreamResource;
import org.springframework.core.io.UrlResource;
import org.springframework.http.MediaType;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.multipart.MultipartFile;

import java.util.List;

@RestController
public class QinwenController3 {

    /**
     * 使用SneakyThrows注解来处理可能抛出的受检异常，避免显式地在方法签名中声明或在方法体内捕获处理
     * 处理多部分表单数据中的文件读取请求，该方法专门用于读取上传的文件内容
     *
     * @param filePath 上传的文件，封装在MultipartFile对象中，用于处理客户端上传的文件数据
     * @return 返回文件内容的字符串表示，如果处理过程中发生异常，则返回null
     */
    @SneakyThrows
    @PostMapping(value = "/readMulipartFile", consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
    public String readMulipartFile(MultipartFile filePath) {
        try {
            //读取IO流文件
            InputStreamResource resource = new InputStreamResource(filePath.getInputStream());
            //使用TikaDocumentReader读取文件内容到Document列表中
            List<Document> documents = new TikaDocumentReader(resource).read();
            //将Document列表转换为内容字符串列表，并以字符串形式返回
            return documents.stream().map(Document::getContent).toList().toString();
```

```
        }catch (Exception e){
            //打印异常堆栈跟踪，用于调试和日志记录
            e.printStackTrace();
        }
        //如果发生异常，返回null
        return "读取失败";
    }

    /**
     * 读取本地文件
     *
     * @return 文件内容的字符串表示
     */
     @SneakyThrows
    @GetMapping(value = "/qinwen3")
    public String readLocalFile() {
        try {
            //使用TikaDocumentReader读取文件内容到Document列表中
            List<Document> documents = new TikaDocumentReader(new
                                FileSystemResource("d:/zmx/hxx.txt")).read();
            //将Document列表中的内容拼接成一个字符串返回
            return documents
                    .stream()
                    .map(Document::getContent)
                    .toList()
                    .toString();
        }catch (Exception e){
            e.printStackTrace();
        }
        return "读取失败";
    }

    /**
     * 读取网络文件
     *
     * @return 文件内容的字符串表示
     */
    @SneakyThrows
    @GetMapping("/readFile2")
    public String readFile2() {
        try {
            return new TikaDocumentReader(new UrlResource("http://www.xxx.com/xx.txt"))
                    .read()
                    .stream()
                    .map(Document::getContent)
                    .toList()
                    .toString();
        }catch (Exception e){
            e.printStackTrace();
        }
        return "读取失败";
    }
}
```

2.1.4.2 转换文档

- `Document`对象是`ETL Pipeline`的核心对象，它包含了文档的元数据和内容。
- 内容转换器：
  - `TokenTextSplitter`：可以把内容切割成更小的块方便`RAG`的时候提升响应速度节省`Token`。
  - `ContentFormatTransformer`：可以把元数据的内容变成键值对字符串。
- 元数据转换器：
  - `SummaryMetadataEnricher`：使用大模型总结文档。会在元数据里面增加一个`summary`字段。
  - `KeywordMetadataEnricher`：使用大模型提取文档关键词。可以在元数据里面增加一个`keywords`字段。
- 从前端传来文件，使用`TikaDocumentReader`读取文件，然后使用`TokenTextSplitter`将文本内容划分成更小的块。
- 案例：

```
package com.hxx.controller;
```

```
import lombok.SneakyThrows;
import org.springframework.ai.document.Document;
import org.springframework.ai.reader.tika.TikaDocumentReader;
import org.springframework.ai.transformer.splitter.TokenTextSplitter;
import org.springframework.core.io.FileSystemResource;
import org.springframework.core.io.InputStreamResource;
import org.springframework.core.io.UrlResource;
import org.springframework.http.MediaType;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.multipart.MultipartFile;

import java.util.List;

@RestController
public class QinwenController4 {

    @PostMapping("/convertFile")
    public String convertFile(MultipartFile file) throws Exception {
        // 从IO流中读取文件
        TikaDocumentReader reader = new TikaDocumentReader(new
InputStreamResource(file.getInputStream()));
        // 将文本内容划分成更小的块
        return new TokenTextSplitter()
                .apply(reader.read()) // 设置每个块的最大长度
                .stream()
                .map(Document::getContent)
                .toList()
                .toString();
    }
}
```

2.1.4.3 存储文档

- 存储到向量数据库或者本地文件中
- 需要先引入向量数据库

```
<dependency>
    <groupId>org.springframework.ai</groupId>
    <artifactId>spring-ai-redis-store</artifactId>
</dependency>
<dependency>
    <groupId>redis.clients</groupId>
    <artifactId>jedis</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.ai</groupId>
    <artifactId>spring-ai-tika-document-reader</artifactId>
</dependency>
```

- 设置docker镜像源: /etc/docker/daemon.json

```
{
  "registry-mirrors": ["https://registry.cn-hangzhou.aliyba.com","https://ccr.ccs.tencentyun.com",
    "https://docker.rainbond.cc",
    "https://elastic.m.daocloud.io",
    "https://elastic.m.daocloud.io",
    "https://docker.m.daocloud.io",
    "https://gcr.m.daocloud.io",
    "https://ghcr.m.daocloud.io",
    "https://k8s-gcr.m.daocloud.io",
    "https://k8s.m.daocloud.io",
    "https://mcr.m.daocloud.io",
    "https://nvcr.m.daocloud.io",
    "https://quay.m.daocloud.io"
]
}
```

- 配置

```
拉取镜像
docker pull redis/redis-stack

查看镜像
docker images redis/redis-stack

创建redis目录
mkdir -p /home/redis/conf
mkdir -p /home/redis/data
cd /home/redis/conf

下载配置文件
wget http://download.redis.io/redis-stable/redis.conf

授权
chmod 777 redis.conf

修改配置文件
vim /home/redis/conf/redis.conf

修改内容
bind 127.0.0.1       注释掉。解除本机绑定
protected-mode yes   改为no。解除本地访问保护模式
requirepass          设置密码
appendonly  yes。    开启aof持久化

运行
docker run --restart=always -d --name redis-stack -p 6379:6379 -p 8001:8001 -v
/home/redis/conf/redis.conf:/redis-stack.conf -v /home/redis/data/:/data redis/redis-stack:latest
```

- 配置连接

```yaml
spring:
  ai:
    vectorstore:
      redis:
        uri: redis://192.168.12.135:6379
  data:
    redis:
      host: 192.168.12.135
      database: 0
      timeout: 10s
      lettuce:
        pool:
          # 连接池最大连接数
          max-active: 200
          # 连接池最大阻塞等待时间（使用负值表示没有限制）
          max-wait: -1ms
          # 连接池中的最大空闲连接
          max-idle: 10
          # 连接池中的最小空闲连接
          min-idle: 0
      repositories:
        enabled: false
```

- 案例

```java
package com.hxx.controller;

import lombok.SneakyThrows;
import org.springframework.ai.document.Document;
import org.springframework.ai.reader.tika.TikaDocumentReader;
import org.springframework.ai.transformer.splitter.TokenTextSplitter;
import org.springframework.ai.vectorstore.VectorStore;
```

```java
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.core.io.InputStreamResource;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.multipart.MultipartFile;

import java.util.List;

@RestController
public class QinwenController5 {

    // 注入VectorStore实例(向量数据库)
    @Autowired
    private VectorStore vectorStore;

    // 读取文件并返回文本内容
    @SneakyThrows
    @PostMapping("/embedding")
    public String readFile(MultipartFile file) {
        try {
            InputStreamResource resource = new InputStreamResource(file.getInputStream());
            // 从IO流中读取文件
            TikaDocumentReader tikaDocumentReader = new TikaDocumentReader(resource);
            // 将文本内容划分成更小的块
            List<Document> splitDocuments = new TokenTextSplitter()
                    // 设置每个块的最大长度
                    .apply(tikaDocumentReader.read());
            // 将划分后的文本块存储在向量数据库中
            vectorStore.add(splitDocuments);
            return "存储在向量数据库中";
        }catch (Exception e){
            e.printStackTrace();
        }
        return "存储失败";
    }
}
```

2.1.5 向量数据库

- VectorStore对象需要提供EmbeddingModel，这个案例提供的是阿里灵积的EmbeddingModel

2.1.5.1 配置向量数据库

- 在redis中创建索引名称

```
FT.CREATE default-index ON HASH PREFIX 1 default: SCHEMA content TEXT WEIGHT 1.0 vector_score NUMERIC SORTABLE
这个命令创建了一个名为 default-index 的索引，用于对带有前缀 default: 的哈希键进行全文搜索，其中 content 是文本字段，vector_score 是数值字段

FT.DROP default-index
删除索引
```

```java
package com.hxx;

import lombok.AllArgsConstructor;
import org.springframework.ai.autoconfigure.vectorstore.redis.RedisVectorStoreAutoConfiguration;
import org.springframework.ai.autoconfigure.vectorstore.redis.RedisVectorStoreProperties;
import org.springframework.ai.embedding.EmbeddingModel;
import org.springframework.ai.vectorstore.RedisVectorStore;
import org.springframework.ai.vectorstore.VectorStore;
import org.springframework.boot.autoconfigure.EnableAutoConfiguration;
import org.springframework.boot.autoconfigure.data.redis.RedisConnectionDetails;
import org.springframework.boot.context.properties.EnableConfigurationProperties;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import redis.clients.jedis.JedisPooled;
```

```
@Configuration
// 禁用SpringAI提供的RedisStack向量数据库的自动配置，会和Redis的配置冲突。
@EnableAutoConfiguration(exclude = {RedisVectorStoreAutoConfiguration.class})
// 读取RedisStack的配置信息
@EnableConfigurationProperties({RedisVectorStoreProperties.class})
@AllArgsConstructor
public class RedisVectorConfig {

    /**
     * 创建RedisStack向量数据库
     *
     * @param embeddingModel 嵌入模型
     * @param properties     redis-stack的配置信息
     * @return vectorStore 向量数据库
     */
    @Bean
    public VectorStore vectorStore(EmbeddingModel embeddingModel,
                                   RedisVectorStoreProperties properties,
                                   RedisConnectionDetails redisConnectionDetails) {
        RedisVectorStore.RedisVectorStoreConfig config =
RedisVectorStore.RedisVectorStoreConfig.builder().withIndexName(properties.getIndex()).withPrefix(properties
.getPrefix()).build();
        return new RedisVectorStore(config, embeddingModel,
                new JedisPooled(redisConnectionDetails.getStandalone().getHost(),
                        redisConnectionDetails.getStandalone().getPort()
                        , redisConnectionDetails.getUsername(),
                        redisConnectionDetails.getPassword()),
                properties.isInitializeSchema());
    }
}
```

2.1.5.2 文档嵌入

- 把文档给`EmbeddingModel`把文本变成向量然后再存入向量数据库。
- 同上例子

2.1.5.3 文档查询

- 调用`vectorStore.similaritySearch(query)`时同样会先把用户的提问给`EmbeddingModel`，将提问变成向量，然后与向量数据库中的文档向量进行相似度计算（cosine值）。
- 要注意：此时向量数据库不会回答用户的提问。
- 案例：

```
package com.hxx.controller;

import lombok.SneakyThrows;
import org.springframework.ai.document.Document;
import org.springframework.ai.reader.tika.TikaDocumentReader;
import org.springframework.ai.transformer.splitter.TokenTextSplitter;
import org.springframework.ai.vectorstore.VectorStore;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.core.io.InputStreamResource;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.multipart.MultipartFile;

import java.util.List;

@RestController
public class QinwenController5 {

    // 注入VectorStore实例(向量数据库)
    @Autowired
    private VectorStore vectorStore;

    // 读取文件并返回文本内容
    @SneakyThrows
```

```
    @PostMapping("/embedding")
    public String readFile(MultipartFile file) {
         // 读取文件内容
        TikaDocumentReader tikaDocumentReader = new TikaDocumentReader(
                new InputStreamResource(file.getInputStream()));
        // 将文件内容拆分为文档
        List<Document> documents = new TokenTextSplitter()
                .apply(tikaDocumentReader.read());
        // 将文档向量存储到向量存储中
        vectorStore.add(documents);
        return "success";
    }

    /**
     * 查询向量数据库
     *
     * @param query 用户的提问
     * @return 匹配到的文档
     */

    @GetMapping("query")
    public List<Document> query(@RequestParam String query) {
        return vectorStore.similaritySearch(query);
    }
}
```

2.1.6 RAG检索增强

- 要搭建自己的知识库除了文档嵌入到向量数据库之外，就是RAG了。当用户提问的时候先从想来数据库搜索相关的资料，再把相关的资料拼接到用户的提问中，再让模型生成答案
- `QuestionAnswerAdvisor`可以在用户发起的提问时，先向数据库查询相关的文档，再把相关的文档拼接到用户的提问中，再让模型生成答案。那就是`RAG`的实现了。
- `RAG`的实现过程：
    1. `query` = 用户的提问； `template` = 提示词模板
    2. `QuestionAnswerAdvisor`会在运行时替换模板中的占位符`question_answer_context`，替换成向量数据库中查询到的文档，即`context=template.replace("question_answer_context",查询到的文档)`。此时的`contextQuery` = `query` + `context`；
    3. `contextQuery`发送给大模型得到答案
- 正常的提问过程是`query`=用户的提问;`query`发送给大模型得到答案。
- 对比`RAG`的提问过程可以发现多了提示词模板和向量数据库查询的过程。
- 案例：

```
package com.hxx.controller;

import lombok.SneakyThrows;
import org.springframework.ai.chat.client.ChatClient;
import org.springframework.ai.chat.client.advisor.QuestionAnswerAdvisor;
import org.springframework.ai.chat.model.ChatModel;
import org.springframework.ai.document.Document;
import org.springframework.ai.reader.tika.TikaDocumentReader;
import org.springframework.ai.transformer.splitter.TokenTextSplitter;
import org.springframework.ai.vectorstore.SearchRequest;
import org.springframework.ai.vectorstore.VectorStore;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.core.io.InputStreamResource;
import org.springframework.http.MediaType;
import org.springframework.http.codec.ServerSentEvent;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.multipart.MultipartFile;
import reactor.core.publisher.Flux;

import java.util.List;
```

```
@RestController
public class QinwenController6 {

    // 注入VectorStore实例(向量数据库)
    @Autowired
    private VectorStore vectorStore;
    @Autowired
    private ChatModel chatModel;

    /**
     * 从向量数据库中查找文档，并将查询的文档作为上下文回答。
     *
     * @param prompt 用户的提问
     * @return SSE流响应
     */
    @GetMapping(value = "/chat/stream/database", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
    public Flux<ServerSentEvent<String>> chatStreamWithDatabase(@RequestParam String prompt) {
        // 1. 定义提示词模板，question_answer_context会被替换成向量数据库中查询到的文档。
        String promptWithContext = """
                下面是上下文信息
                ---------------------
                {question_answer_context}
                ---------------------
                给定的上下文和提供的历史信息，而不是事先的知识，回复用户的意见。如果答案不在上下文中，告诉用户你不能回答
这个问题。
                """;
        Flux<ServerSentEvent<String>> message = ChatClient.create(chatModel).prompt()
                .user(prompt)
                // 2. QuestionAnswerAdvisor会在运行时替换模板中的占位符`question_answer_context`，替换成向量数
据库中查询到的文档。此时的query=用户的提问+替换完的提示词模板;
                .advisors(new QuestionAnswerAdvisor(vectorStore, SearchRequest.defaults(),
promptWithContext))
                .stream()
                // 3. query发送给大模型得到答案
                .content()
                .map(chatResponse -> ServerSentEvent.builder(chatResponse)
                        .event("message")
                        .build());
        System.out.println("---------->>>message:" + message);
        return message;
    }
}
```

2.1.7 FunctionCall

- 编写函数的步骤
  1. 定义函数的入参和返回结果
  2. 实现 `java.util.function.Function` 表示它的一个可以被AI调用的函数
  3. 描述函数，SpringAi会自动将下面的Java注解转换成

     function call

     格式的描述

     - `function.name`： `@Service` 将该类注册为 spring bean，bean的名字会作为函数的名称。
     - `function.description`：`@Description` 可以描述函数的用途
     - `function.parameters.properties.xxx`：`@JsonProperty` 描述参数的属性名称
     - `function.parameters.required`：`@JsonProperty` 是否必填
     - `function.parameters.properties.xxx.description`：`@JsonPropertyDescription` 描述参数的作用
     - `function.parameters.properties.xxx.type`：String类型会自动生成JSON Scheme reference下的 `string` 类型，其他的 `Integer`，`List` 等类型也一样。
- 案例：

```
package com.hxx.utils;

import com.fasterxml.jackson.annotation.JsonProperty;
import com.fasterxml.jackson.annotation.JsonPropertyDescription;
import lombok.Data;
```

```java
import lombok.SneakyThrows;
import lombok.extern.slf4j.Slf4j;
import org.springframework.ai.reader.tika.TikaDocumentReader;
import org.springframework.context.annotation.Description;
import org.springframework.core.io.FileSystemResource;
import org.springframework.stereotype.Service;

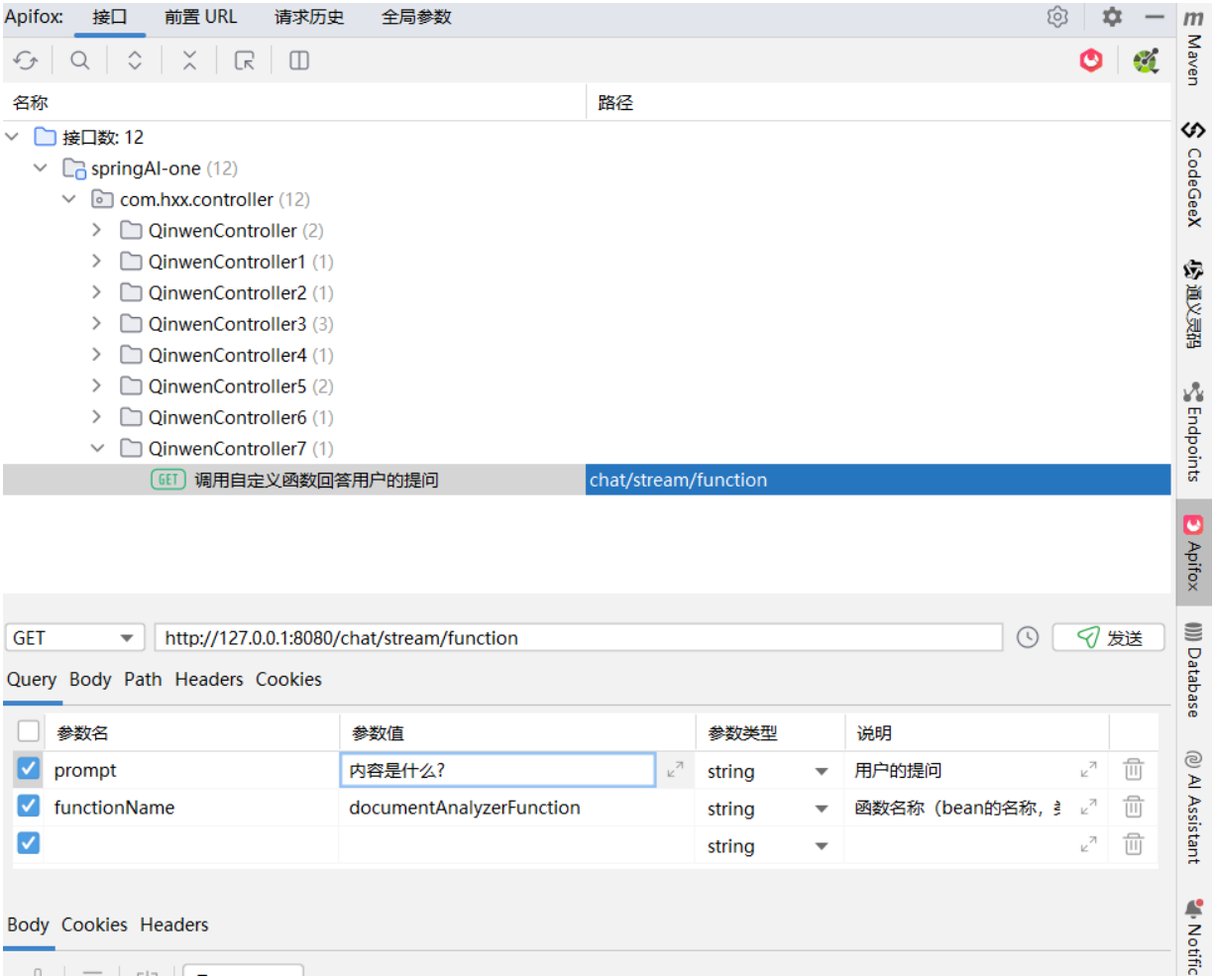

import java.util.function.Function;

/**
 * 通过@Description描述函数的用途，这样ai在多个函数中可以根据描述进行选择。
 */
@Description("文档解析函数")
@Service
@Slf4j
public class DocumentAnalyzerFunction
        implements Function<DocumentAnalyzerFunction.Request,DocumentAnalyzerFunction.Response> {
    /**
     * 通过@JsonProperty声明属性名称和是否必填
     * 通过@JsonPropertyDescription描述属性的用途，这样ai可以提取出符合参数描述的内容。
     */
    @Data
    public static class Request {
        @JsonProperty(required = true, value = "path")
        @JsonPropertyDescription(value = "需要解析的文档路径")
        String path;
    }

    public record Response(String result) {
    }

    @SneakyThrows
    @Override
    public Response apply(Request request) {
        // ai解析用户的提问得到path参数，使用tika读取本地文件获取内容。把读取到的内容再返回给ai作为上下文去回答用户的问题。
        TikaDocumentReader tikaDocumentReader = new TikaDocumentReader(new FileSystemResource(request.path));
        return new Response(tikaDocumentReader.read().get(0).getContent());
    }
}
```

- 使用函数
  - 在实际的开发中可以接收多个函数，通过 `functions` 参数传入。然后ai会根据提问从这些函数中选择一个执行。
  - 调用：
    - ```
      GET http://localhost:9902/message/chat/stream/function?prompt=D:\简历.pdf，这份简历的亮点是什么？&functionName=documentAnalyzerFunction
      ```
  - 案例：
      ```



2.2 聊天案例

2.2.1 创建数据表

- 后端实现会话和消息记录必然要先从建表开始，然后编写增删改查，最后实现一些功能。

  会话

  - 会话保存接口
    - 会话保存dto
  - 会话列表接口
    - 获取当前登录用户id作为查询条件
    - 一对多关联获取会话内的消息
  - 会话删除接口
    - 根据会话id列表批量删除
    - 删除会话时，会话内的消息也一并级联删除
  - 会话id查询接口
    - 一对多关联获取会话内的消息
- 消息
  - 消息发送接口
    - 消息保存dto（接收前端发送的消息）
    - 消息实体内容设计，支持图片/语音/文字
    - `MessageChatMemoryAdvisor` 历史消息增强和消息自动保存，调用 `AiMessageChatMemory` 的 `get` 方法获取会话的历史消息和 `add` 方法保存消息。
    - 使用 `ChatClient` 填写用户发送的消息（可能包含图片/语言），填写 `MessageChatMemoryAdvisor`。最后发起流式请求，获取AI回复的消息。
  - 历史消息实现
    - `add` 方法，保存用户发送的消息和AI回复的消息。因为需要填写创建人id，创建人id是在请求里面存储的。保存的时候是在异步的线程里面，要模拟请求环境才能插入数据到数据库。
    - `get` 方法，获取会话历史消息

- `clear`方法，清空会话的历史消息
- 消息转换器，将消息实体转换为SpringAI的`Message`，将`Message`转换为消息实体。互相转换

2.2.2 实体

- BaseEntity

```
@Data
public class BaseEntity {
    @TableId(type=IdType.ASSIGN_UUID)
    private String id;

    @TableField(fill = FieldFill.INSERT)
    private LocalDateTime createdTime;

    @TableField(fill = FieldFill.INSERT_UPDATE)
    private LocalDateTime editedTime;

    private String creatorId;

    private String editorId;
}
```

- 消息表

```
create table ai_message
(
    id  varchar(36) not null primary key comment '主键',
    created_time datetime(6) not null comment '创建时间',
    edited_time  datetime(6) not null comment '更新时间',
    creator_id   varchar(32) not null comment '创建者',
    editor_id    varchar(32) not null comment '修改者',
    type          varchar(32) not null comment '消息类型(用户/ 助手/ 系统)',
    text_content  text        not null comment '消息内容',
    medias        json        null comment '媒体内容如图片链接、语音链接',
    ai_session_id varchar(32) not null comment '会话id'
);
```

- 消息实体类

```
@Data
@AllArgsConstructor
@NoArgsConstructor
@Accessors(chain = true)
public class MediaZmx {
    public String type = "image";
    public String data = "000";
}
```

```
/**
 * 历史消息
 */
@Data
@TableName("ai_message")
public class AiMessage extends BaseEntity {

    /**
     * 消息类型(用户/助手/系统)
     */
    private MessageType type;

    /**
     * 消息内容
     */
    private String textContent;

    @TableField(typeHandler = FastjsonTypeHandler.class)
```

```
    private List<MediaZmx> medias;

    @TableField("ai_session_id")
    private String sessionId;
}
```

- 会话实体

```
create table ai_session
(
    id  varchar(36) not null primary key comment '主键',
    created_time datetime(6) not null comment '创建时间',
    edited_time  datetime(6) not null comment '更新时间',
    creator_id   varchar(32) not null comment '创建者',
    editor_id    varchar(32) not null comment '修改者',
    name         varchar(32) not null comment '会话名称'
);
```

- 会话实体

```
@Data
@TableName("ai_session")
public class AiSession extends BaseEntity{
    /**
     * 会话名称
     */
    private String name;

    private List<AiMessage> messages;

    private String sessionId;
}
```

2.2.3 AiMessageChatMemory

- `AiMessageChatMemory` 实现了 `ChatMemory` 接口，用于保存聊天记录到数据库.
- 案例:

```
package com.hxx.common;

import cn.hutool.core.collection.CollectionUtil;
import com.baomidou.mybatisplus.core.conditions.query.LambdaQueryWrapper;
import com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
import com.hxx.entity.AiMessage;
import com.hxx.entity.MediaZmx;
import com.hxx.mapper.AiMessgeMapper;
import lombok.AllArgsConstructor;
import lombok.SneakyThrows;
import org.springframework.ai.chat.memory.ChatMemory;
import org.springframework.ai.chat.messages.*;
import org.springframework.ai.model.Media;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.MediaType;
import org.springframework.stereotype.Service;

import java.net.MalformedURLException;
import java.net.URL;
import java.util.ArrayList;
import java.util.List;
/*

可以把AiMessageChatMemory注入到MessageChatMemoryAdvisor中。
MessageChatMemoryAdvisor的作用有下面三个
    用户消息发送给大模型之前，获取会话内的最新n条记录和用户的消息拼接在一起，形成历史消息记录。
    拼接完历史消息之后把用户发送的消息保存到数据库。
    大模型回复完消息之后，将回复的消息保存到数据库。
    ----------------------------------------
```

```
    // 注入chatMemory
    private final AiMessageChatMemory chatMemory;
   // 传入chatMemory，会话id，查询最近n条历史消息
   var messageChatMemoryAdvisor = new MessageChatMemoryAdvisor(chatMemory, input.getSessionId(), 10);

 */
@Service
@AllArgsConstructor
public class AiMessageChatMemory  implements ChatMemory{

    @Autowired
    private AiMessgeMapper messageRepository;

    /**
     * 不实现，手动前端发起请求保存用户的消息和大模型回复的消息
     */
    @Override
    public void add(String conversationId,
                    List<org.springframework.ai.chat.messages.Message> messages) {

    }

    /**
     * 查询会话内的消息最新n条历史记录
     *
     * @param conversationId 会话id
     * @param lastN          最近n条
     * @return org.springframework.ai.chat.messages.Message格式的消息
     */
    @Override
    public List<Message> get(String conversationId, int lastN) {
        LambdaQueryWrapper<AiMessage> wrapper = new LambdaQueryWrapper<>();
        wrapper.eq(AiMessage::getSessionId, conversationId);
        wrapper.orderByDesc(AiMessage::getCreatedTime);
        wrapper.last("limit " + lastN);

        if (CollectionUtil.isNotEmpty(messageRepository.selectList(wrapper))) {
            return messageRepository.selectList(wrapper)
                    .stream()
                    .map(AiMessageChatMemory::toSpringAiMessage)
                    .toList();
        }
        return null;
    }

    /**
     * 清除会话内的消息
     *
     * @param conversationId 会话id
     */
    @Override
    public void clear(String conversationId) {
        QueryWrapper<AiMessage> wrapper = new QueryWrapper<>();
        wrapper.eq("session_id", conversationId);
        messageRepository.delete(wrapper);
    }

    /**
     * 将自定义的AiMessage对象转换为Spring AI框架中的Message对象
     * 此方法支持三种消息类型：Assistant、User和System的转换
     * 对于用户消息，如果包含媒体文件，将一并转换
     *
     * @param aiMessage 自定义的AiMessage对象，包含消息的基本信息和内容
     * @return 返回转换后的Spring AI框架中的Message对象
     * @throws RuntimeException 如果遇到不支持的消息类型，抛出运行时异常
     */
    @SneakyThrows
    public static org.springframework.ai.chat.messages.Message toSpringAiMessage(AiMessage
                                                                                  aiMessage) {
        // 初始化媒体文件列表
        List<Media> mediaList = new ArrayList<>();
        // 如果消息中包含媒体文件，则转换每个媒体文件为Spring AI框架中的Media对象
```

```
        if (!CollectionUtil.isEmpty(aiMessage.getMedias())) {
            mediaList =
                aiMessage.getMedias().stream().map(AiMessageChatMemory::toSpringAiMedia).toList();
        }
        // 根据消息类型，创建对应的Message对象
        if (aiMessage.getType().equals(MessageType.ASSISTANT)) {
            // 如果是助理消息，创建AssistantMessage对象
            return new AssistantMessage(aiMessage.getTextContent());
        }
        if (aiMessage.getType().equals(MessageType.USER)) {
            // 如果是用户消息，创建UserMessage对象，并包含媒体文件
            return new UserMessage(aiMessage.getTextContent(), mediaList);
        }
        if (aiMessage.getType().equals(MessageType.SYSTEM)) {
            // 如果是系统消息，创建SystemMessage对象
            return new SystemMessage(aiMessage.getTextContent());
        }
        // 如果是不支持的消息类型，抛出异常
        throw new RuntimeException("不支持的消息类型");
    }

    /**
     * 将MediaZmx对象转换为Media对象
     * 此方法使用了MediaZmx类型的media参数，将其type和data属性分别转换为MediaType和URL对象，
     * 并用这些对象来创建一个新的Media对象如果media的data属性不是有效的URL格式，
     * 则抛出运行时异常
     *
     * @param media MediaZmx对象，包含type和data属性，用于创建Media对象
     * @return 返回一个新的Media对象，其类型和数据URL根据media参数的type和data属性创建
     * @throws RuntimeException 如果media的data属性不是有效的URL格式，则抛出此异常
     */
    @SneakyThrows
    public static Media toSpringAiMedia(MediaZmx media)  {
        try {
            // 创建一个新的Media对象，使用MediaZmx的type属性创建MediaType对象，data属性创建URL对象
            return new Media(new MediaType(media.getType()), new URL(media.getData()));
        }catch (MalformedURLException e){
            // 如果media的data属性不是有效的URL格式，抛出运行时异常
            throw new RuntimeException("url格式错误");
        }
    }
}
```

- 可以把`AiMessageChatMemory`注入到`MessageChatMemoryAdvisor`中。

  MessageChatMemoryAdvisor的作用有下面三个

  1. 用户消息发送给大模型之前，获取会话内的最新n条记录和用户的消息拼接在一起，形成历史消息记录。
  2. 拼接完历史消息之后把用户发送的消息保存到数据库。
  3. 大模型回复完消息之后，将回复的消息保存到数据库。
  4. 案例代码：

  ```
  // 注入chatMemory
  private final AiMessageChatMemory chatMemory;
  // 传入chatMemory，会话id，查询最近n条历史消息
  var messageChatMemoryAdvisor = new MessageChatMemoryAdvisor(chatMemory,
                                                              input.getSessionId(),
                                                              10);
  ```

- 发送消息
  - 通常来说发送消息给大模型只需要填写用户的消息就行，但是要支持历史会话需要获取历史消息。这个功能可以使用MessageChatMemoryAdvisor来实现。

```
package com.hxx.controller;

import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.databind.ObjectMapper;
```

```java
import com.hxx.common.AiMessageChatMemory;
import com.hxx.entity.AiMessage;
import com.hxx.mapper.AiMessgeMapper;
import com.hxx.mapper.AiSessionMapper;
import lombok.SneakyThrows;
import org.springframework.ai.chat.client.ChatClient;
import org.springframework.ai.chat.client.advisor.MessageChatMemoryAdvisor;
import org.springframework.ai.chat.messages.Message;
import org.springframework.ai.chat.messages.UserMessage;
import org.springframework.ai.chat.model.ChatModel;
import org.springframework.ai.chat.model.ChatResponse;
import org.springframework.ai.model.Media;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.ApplicationContext;
import org.springframework.http.MediaType;
import org.springframework.http.codec.ServerSentEvent;
import org.springframework.util.CollectionUtils;
import org.springframework.web.bind.annotation.*;
import reactor.core.publisher.Flux;

@RequestMapping("message")
@RestController
public class AiMessageController {
    @Autowired
    private AiMessageChatMemory chatMemory;
    @Autowired
    private ChatModel dashScopeAiChatModel;
    @Autowired
    private  ObjectMapper objectMapper;
    @Autowired
    private AiMessgeMapper messageRepository;

    @Autowired
    private ApplicationContext applicationContext;
    @Autowired
    private AiSessionMapper aiSessionRepository;

    /**
     * 删除与特定会话相关的聊天历史记录
     * 此方法通过DeleteMapping注解映射到HTTP DELETE请求，请求路径为"/history/{sessionId}"
     * 它接受一个路径变量sessionId，表示要删除聊天历史记录的会话ID
     *
     * @param sessionId 会话ID，用于标识和删除特定会话的聊天历史记录
     */
    @DeleteMapping("history/{sessionId}")
    public void deleteHistory(@PathVariable String sessionId) {
        chatMemory.clear(sessionId);
    }

    /**
     * 消息保存
     *
     * @param input 用户发送的消息/AI回复的消息
     */
    @PostMapping
    public void save(@RequestBody AiMessage input) {
        messageRepository.insert(input);
    }

    /**
     * @param input 消息包含文本信息，会话id，多媒体信息（图片语言）。
     * @return SSE流
     */
    @PostMapping(value = "chat", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
    public Flux<ServerSentEvent<String>> chat(@RequestBody AiMessage input) {
        // MessageChatMemoryAdvisor的三个参数解释。
        // 1. 如果需要存储会话和消息到数据库，自己可以实现ChatMemory接口，这里使用自己实现的AiMessageChatMemory，数
据库存储。
        // 2. 传入会话id，MessageChatMemoryAdvisor会根据会话id去查找消息。
        // 3. 只需要携带最近10条消息
        var messageChatMemoryAdvisor = new MessageChatMemoryAdvisor(chatMemory, input.getSessionId(),
10);
```

```
        // 获取ChatClient
        return ChatClient.create(dashScopeAiChatModel).prompt()
                .user(promptUserSpec -> {
                    Message message = AiMessageChatMemory.toSpringAiMessage(input);
                    if (message instanceof UserMessage userMessage &&
 !CollectionUtils.isEmpty(userMessage.getMedia())) {
                        // 用户发送的图片/语言
                        Media[] medias = new Media[userMessage.getMedia().size()];
                        promptUserSpec.media(userMessage.getMedia().toArray(medias));
                    }
                    // 用户发送的文本
                    promptUserSpec.text(message.getContent());
                })
                .advisors(messageChatMemoryAdvisor)
                .stream()
                .content()
                .map(chatResponse -> ServerSentEvent.builder(chatResponse)
                        .event("message")
                        .build());
    }
    @SneakyThrows
    public String toJson(ChatResponse response) {
        try {
            return objectMapper.writeValueAsString(response);
        } catch (JsonProcessingException e) {
            e.printStackTrace();
        }
        return null;
    }
}
```

2.3 图片处理（待，必须openAI）

2.3.1 配置

```
dashscope:
  api-key: sk-xxxxx
  chat:
    options:
      model: qwen-v1-plus
```

2.3.2 图片上传

```
package com.hxx.common.upload;

import com.hxx.common.util.R;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.multipart.MultipartFile;
import java.io.File;
import java.io.IOException;
import java.util.UUID;

@RestController
@RequestMapping("/upload")
public class UploadFile {

    @PostMapping
    public R upload(MultipartFile file) throws IOException {
        //把文件的内容存储到本地磁盘上
        String originFileName = file.getOriginalFilename();
        //需要保证文件名字唯一，防止文件被覆盖
        String fileName = UUID.randomUUID().toString().replace("-","")
                +originFileName.substring(originFileName.lastIndexOf("."));
        String path = "D:/tupian/files/";
        File dest = new File(path);
        //创建文件夹
```

```
        if (!dest.exists()){
            dest.mkdirs();
        }
        file.transferTo(new File(path+fileName));
        return R.ok().data("url",path+fileName);
    }
}
```

```
@Configuration
public class WebMvcConfig implements WebMvcConfigurer {

    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        String path = "d:/tupian/files/";
        registry.addResourceHandler("/tupian/files/**")
                .addResourceLocations("file:" + path);
    }
```